However, the WORKDIR instruction can help us fix the situation. Effectively, it performs a *change directory* (cd) command, changing what the image considers its current directory. The next line in our Dockerfile uses it to set /usr/src/app as the working directory:

```
WORKDIR /usr/src/app
```

Now running bin/rails server (and similar) commands will work because they will be executed from the correct directory.

You can use multiple WORKDIR instructions in your Dockerfile, each one remaining in effect until another one is issued. The final WORKDIR will be the initial working directory for containers created from the image.

Finally, we come to the last line of our Dockerfile:

```
RUN bundle install
```

The command is executed from the container's current working directory, which in the previous command was set to be /usr/src/app. So this will install the gems defined in our Rails project's Gemfile, which are needed in order to start the application.

Putting It All Together

Armed with all this knowledge, our Dockerfile should now be much more understandable. Let's review it one more time:

```
Line 1  FROM ruby:2.6
    2
    3  RUN apt-get update -yqq
    4  RUN apt-get install -yqq --no-install-recommends nodejs
    5
    6  COPY . /usr/src/app/
    7
    8  WORKDIR /usr/src/app
    9  RUN bundle install
```

First, on line 1, we say that our custom image will use the ruby:2.6 image as its starting point. Next we update the apt package manager's package information (line 3), so it knows where to install things from. Then we use it to install nodejs (line 4), which we need for Rails' asset pipeline.

With the prerequisites for Rails taken care of, we then copy our Rails app files from our local directory into the container at /usr/src/app (line 6) so they are baked into the image. We make this the current working directory for the image (line 8) so that we can execute Rails commands against the image from the correct directory.